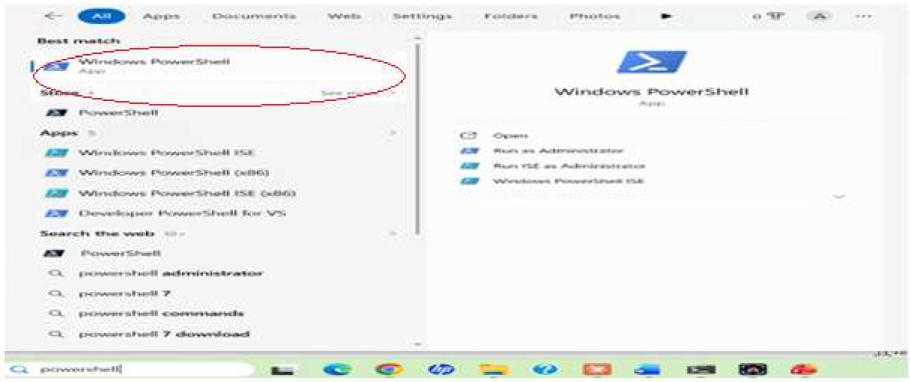
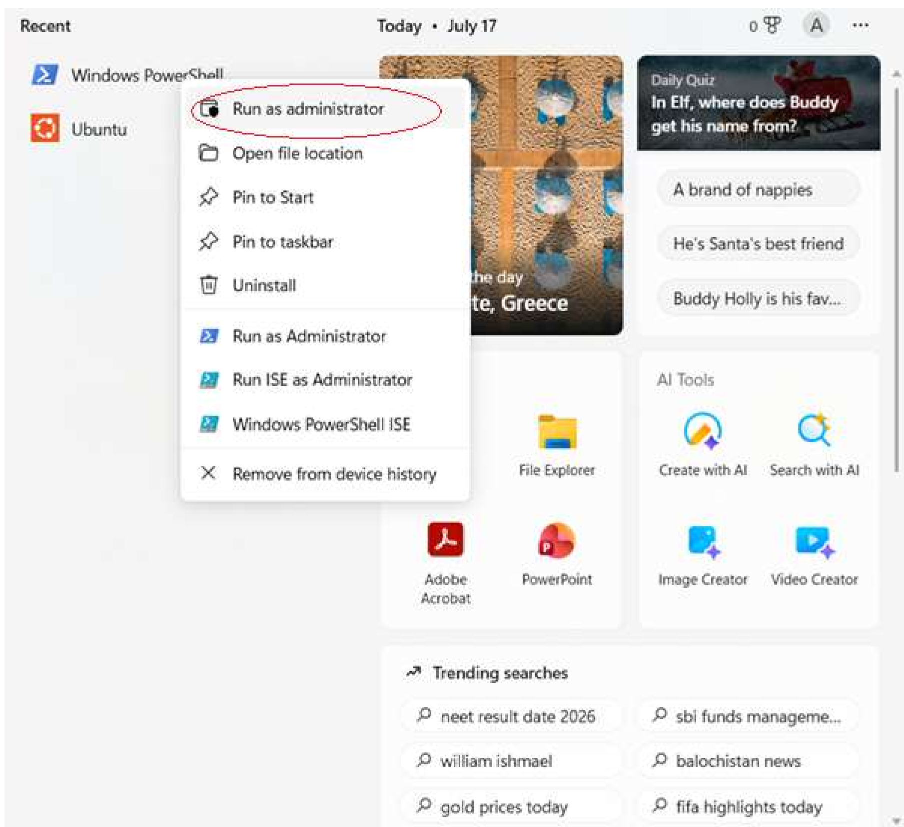
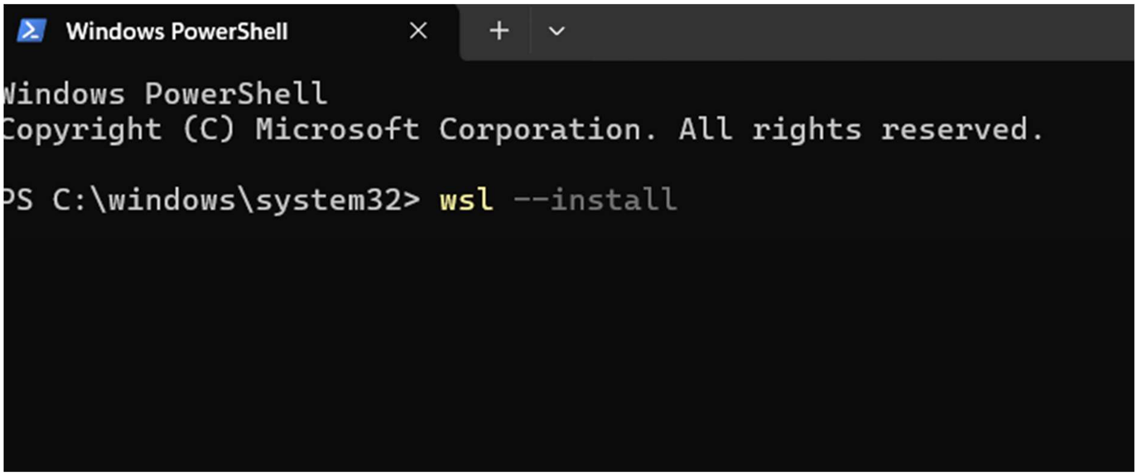
Step 1: Open PowerShell as Administrator

1. Press **Windows**.
2. Search for **PowerShell**.



3. Right-click **Windows PowerShell**.





Step 3: Restart Your Computer

After installation, restart your PC when prompted.

Step 4: Complete Ubuntu Setup

After restarting:

1. Ubuntu opens automatically (or search for **Ubuntu** in the Start menu).
2. Wait for the installation to finish.
3. Create a Linux username.

Example:

Enter new UNIX username: anusha

Step 5: Create a Password

You'll then be prompted to create a password.

New password: ****

Retype new password:****

Step 6: Update Ubuntu

```
hemasai@Hemasai:/mnt/c/WINDOWS/system32$ sudo apt update
[sudo: authenticate] Password:
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [417 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [97.2 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.7 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [159 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [51.2 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.2 kB]
Get:10 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [349 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/restricted Translation-en [67.8 kB]
Hit:12 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [492 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [118 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [97.8 kB]
Get:16 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [243 kB]
Get:17 http://archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [81.0 kB]
Get:18 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [184 kB]
Get:19 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [367 kB]
Get:20 http://archive.ubuntu.com/ubuntu resolute-updates/restricted Translation-en [70.8 kB]
Get:21 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Packages [11.3 kB]
Get:22 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse Translation-en [3032 B]
Fetched 3173 kB in 3s (1080 kB/s)
123 packages can be upgraded. Run 'apt list --upgradable' to see them.
hemasai@Hemasai:/mnt/c/WINDOWS/system32$
```

Step 7: Install GCC (C Compiler)

```
123 packages can be upgraded. Run 'apt list --upgradable' to see them.
hemasai@Hemasai:/mnt/c/WINDOWS/system32$ sudo apt install build-essential
Upgrading:
  build-essential

Installing dependencies:
  libcrypt-dev

Summary:
  Upgrading: 1, Installing: 1, Removing: 0, Not Upgrading: 122
  Download size: 0 B / 127 kB
  Space needed: 384 kB / 1024 GB available

Continue? [Y/n] y
Selecting previously unselected package libcrypt-dev:amd64.
(Reading database ... 42307 files and directories currently installed.)
Preparing to unpack .../libcrypt-dev_1%3a4.5.1-1_amd64.deb ...
Unpacking libcrypt-dev:amd64 (1:4.5.1-1) ...
Preparing to unpack .../build-essential_12.12ubuntu2.26.04.2_amd64.deb ...
Unpacking build-essential (12.12ubuntu2.26.04.2) over (12.12ubuntu2) ...
Setting up libcrypt-dev:amd64 (1:4.5.1-1) ...
Setting up build-essential (12.12ubuntu2.26.04.2) ...
Processing triggers for man-db (2.13.1-1build1) ...
```

Step 8: Verify the Installation

```
Processing triggers for man-db (2.13.1-1build1) ...
hemasai@Hemasai:/mnt/c/WINDOWS/system32$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

hemasai@Hemasai:/mnt/c/WINDOWS/system32$
```

Step 9: Create Your C Program

Example: nano process.c

Step 10: Compile the Program

```
hemasai@Hemasai:/mnt/c/WINDOWS/system32$ nano process.c
hemasai@Hemasai:/mnt/c/WINDOWS/system32$ gcc process.c -o process
hemasai@Hemasai:/mnt/c/WINDOWS/system32$ ./process
Parent Process
Parent PID : 955
Child PID  : 956

Child Process
Child PID : 956
Parent PID: 955

Enter a Linux command: pwd
/mnt/c/WINDOWS/system32
Child process completed.
hemasai@Hemasai:/mnt/c/WINDOWS/system32$
```

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;
    char command[100];

    printf("Parent Process\n");

    pid = fork();

    if (pid < 0)
    {
        perror("Fork failed");
        return 1;
    }

    if (pid > 0)
    {
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n\n", pid);

        wait(NULL);

        printf("Child process completed.\n");
    }
    else
    {
        printf("Child Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n\n", getppid());

        printf("Enter a Linux command: ");
        scanf("%99s", command);
```

[Read 44 lines]

```c
int main()
{
    pid_t pid;
    char command[100];

    printf("Parent Process\n");

    pid = fork();

    if (pid < 0)
    {
        perror("Fork failed");
        return 1;
    }

    if (pid > 0)
    {
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n\n", pid);

        wait(NULL);

        printf("Child process completed.\n");
    }
    else
    {
        printf("Child Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n\n", getppid());

        printf("Enter a Linux command: ");
        scanf("%99s", command);

        system(command);
    }

    return 0;
}
```

```
^G Help        ^O Write Out   ^F Where Is    ^K Cut         ^T Execute     ^C Locatio
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To L
```

```
childprocess    file1.txt  hello       hemasaiprograms  parentandchild.c
childprocess.c  file2.txt  hello.c     input.txt        parentchild
hemasai@Hemasai:~$ nano exec.c
hemasai@Hemasai:~$ gcc exec.c -o exec
hemasai@Hemasai:~$ ./exec
Before exec()
total 448
drwxr-xr-x 2 hemasai hemasai  4096 Aug  4 07:10 Documents
drwxr-xr-x 2 hemasai hemasai  4096 Jul 28 04:07 Hemasai
drwxr-xr-x 2 hemasai hemasai  4096 Aug  4 07:12 OSLab
-rw-r--r-- 1 hemasai hemasai     6 Aug  4 12:10 a.txt
-rw-r--r-- 1 hemasai hemasai     7 Aug  4 12:10 b.txt
-rwxr-xr-x 1 hemasai hemasai 16088 Aug 14 09:48 childprocess
-rw-r--r-- 1 hemasai hemasai   233 Aug 14 09:48 childprocess.c
-rw-r--r-- 1 hemasai hemasai    18 Aug  4 12:11 data.txt
-rwxr-xr-x 1 hemasai hemasai 15992 Aug 14 10:12 exclp
-rw-r--r-- 1 hemasai hemasai   161 Aug 14 10:11 exclp.c
-rwxr-xr-x 1 hemasai hemasai 16040 Aug 24 05:29 exec
-rw-r--r-- 1 hemasai hemasai   166 Aug 24 05:28 exec.c
-rw-r--r-- 2 hemasai hemasai    12 Aug  4 12:01 file1.txt
-rw-r--r-- 2 hemasai hemasai    12 Aug  4 12:01 file2.txt
-rw-r--r-- 1 hemasai hemasai    80 Jul 31 09:33 filename.c
-rwxr-xr-x 1 hemasai hemasai 15992 Aug  5 03:56 fork
-rw-r--r-- 1 hemasai hemasai   104 Aug  5 03:56 fork.c
-rwxr-xr-x 1 hemasai hemasai 16080 Aug  5 04:01 forkexec
-rw-r--r-- 1 hemasai hemasai   269 Aug  5 04:01 forkexec.c
-rwxr-xr-x 1 hemasai hemasai 15952 Aug  5 03:52 hello
-rw-r--r-- 1 hemasai hemasai    80 Aug  5 03:52 hello.c
-rwxr-xr-x 1 hemasai hemasai 15952 Aug  4 03:49 hello1
-rw-r--r-- 1 hemasai hemasai    99 Aug  4 03:49 hello1.c
-rw-r--r-- 1 hemasai hemasai    76 Aug  4 16:15 hemasai
-rwxr-xr-x 1 hemasai hemasai 15960 Aug  4 16:19 hemasaifile
-rw-r--r-- 1 hemasai hemasai    76 Aug  4 16:18 hemasaifile.c
-rwxr-xr-x 1 hemasai hemasai 15952 Jul 31 09:34 hemasaiprograms
-rw-r--r-- 1 hemasai hemasai    30 Aug  7 09:30 input.txt
-rwxr-xr-x 1 hemasai hemasai 16056 Aug 14 10:08 multiplechildprocess
-rw-r--r-- 1 hemasai hemasai   166 Aug 14 10:08 multiplechildprocess.c
-rw-r--r-- 1 hemasai hemasai     0 Aug  4 12:03 new.txt
-rw-r--r-- 1 hemasai hemasai    30 Aug  7 09:30 output.txt
-rwxr-xr-x 1 hemasai hemasai 16088 Aug 14 10:17 parentandchild
-rw-r--r-- 1 hemasai hemasai   335 Aug 14 10:16 parentandchild.c
-rwxr-xr-x 1 hemasai hemasai 16000 Aug  5 03:57 parentchild
-rw-r--r-- 1 hemasai hemasai   234 Aug  5 03:57 parentchild.c
-rwxr-xr-x 1 hemasai hemasai 16072 Aug 14 10:00 pfc
-rw-r--r-- 1 hemasai hemasai   262 Aug 14 09:59 pfc.c
-rwxr-xr-x 1 hemasai hemasai 15992 Aug  5 03:55 pid
-rw-r--r-- 1 hemasai hemasai   111 Aug  5 03:55 pid.c
-rwxr-xr-x 1 hemasai hemasai 16088 Aug  5 03:57 piddisplay
-rw-r--r-- 1 hemasai hemasai   294 Aug  5 03:57 piddisplay.c
-rwxr-xr-x 1 hemasai hemasai 16000 Aug 14 09:38 processid
-rw-r--r-- 1 hemasai hemasai   113 Aug 14 09:37 processid.c
-rw-r--r-- 1 hemasai hemasai    30 Aug  7 09:33 sample.txt
-rwxr-xr-x 1 hemasai hemasai     0 Aug  4 12:24 script.sh
lrwxrwxrwx 1 hemasai hemasai     9 Aug  4 12:25 shortcut -> file1.txt
-rwxr-xr-x 1 hemasai hemasai 16304 Aug  7 09:20 task1
-rw-r--r-- 1 hemasai hemasai   557 Aug  7 09:20 task1.c
-rwxr-xr-x 1 hemasai hemasai 16168 Aug  7 09:29 task2
-rw-r--r-- 1 hemasai hemasai   532 Aug  7 09:28 task2.c
-rwxr-xr-x 1 hemasai hemasai 16056 Aug  5 03:53 username
-rw-r--r-- 1 hemasai hemasai   160 Aug  5 03:53 username.c
-rwxr-xr-x 1 hemasai hemasai 16032 Aug  5 03:58 wait
-rw-r--r-- 1 hemasai hemasai   277 Aug  5 03:58 wait.c
hemasai@Hemasai:~$
```

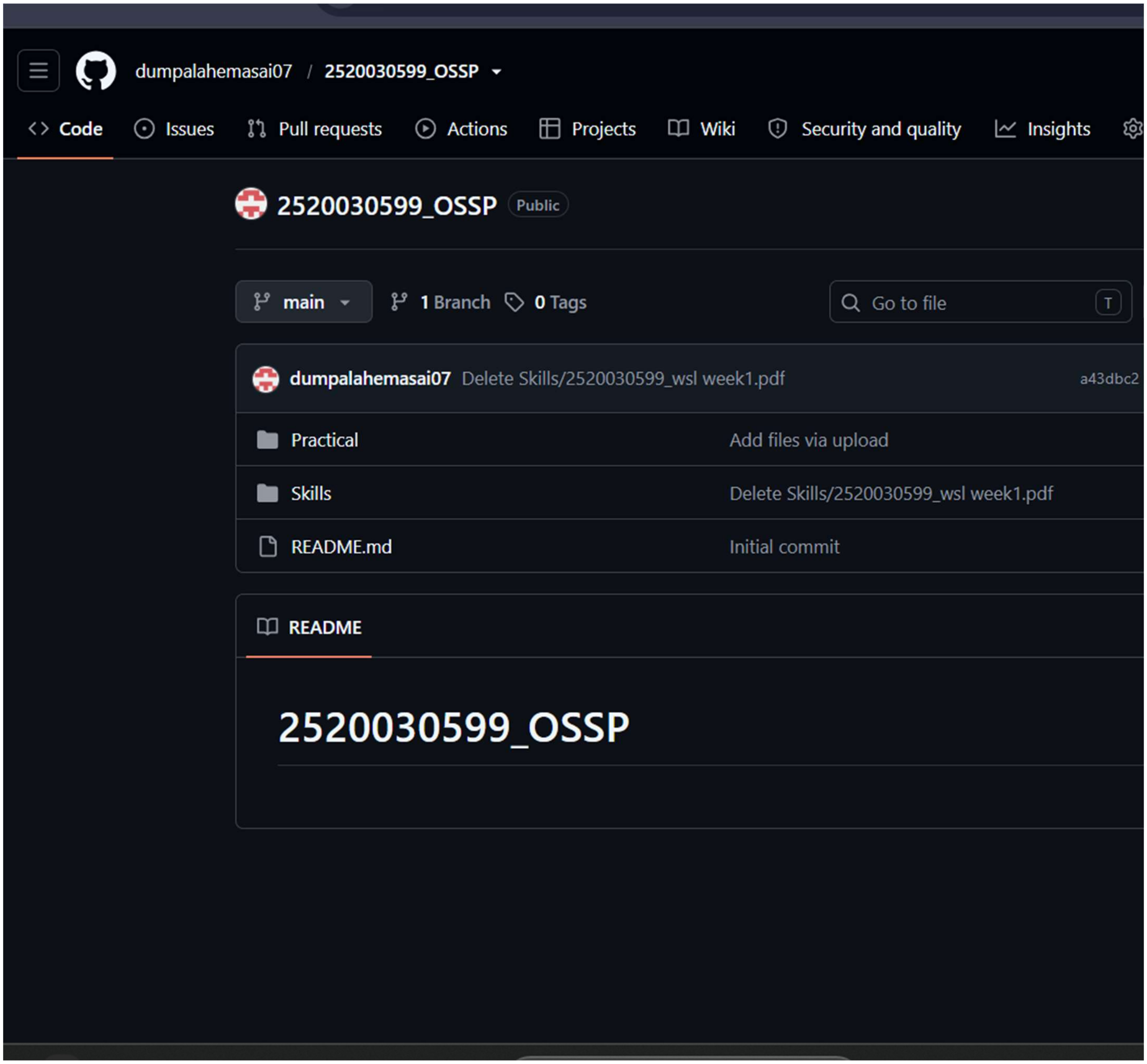
hemasai@Hemasai: ~

```
  GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Before exec()\n");

    execlp("ls", "ls", "-l", NULL);

    perror("exec failed");

    return 1;
}
```

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid;

    printf("Before fork()\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID  : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);
    }

    return 0;
```

```c
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid;

    printf("Before fork()\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID  : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);
    }

    return 0;
}
```

```
compilation terminated.
hemasai@Hemasai:~$ gcc fork.c -o fork
hemasai@Hemasai:~$ ./fork
Before fork()
Parent Process
Parent PID : 6804
Child PID  : 6805
Child Process
Child PID  : 6805
Parent PID : 1109
hemasai@Hemasai:~$
```



https://github.com/dumpalahemasai07/2520030599_OSSP